

راهنمای ارائه پروژه - مصاحبه React/Next.js

پروژه چیست؟

شهر امید - یک showroom مجازی مبلمان با قابلیت‌های پیشرفته:

- مشاهده 3D محصولات با تغییر رنگ real-time
- پیاده‌روی در فروشگاه مجازی (first-person)
- پیش‌نمایش AR در موبایل
- Configurator ماشین (bonus feature)

Next.js 14 • React 18 • TypeScript • Three.js • Zustand • Tailwind 4

معماری کلی پروژه 📁

ساختار اصلی

app/	→ Pages (Next.js App Router)
components/	→ React Components
lib/	→ Business Logic & Utils
stores/	→ Zustand Stores
hooks/	→ Custom Hooks
contexts/	→ React Contexts
public/	→ Static Assets (models, configs)

تصمیم معماری: جداسازی مسئولیت‌ها

چرا این ساختار؟

- **فقط UI rendering** components/
- **تمام business logic و calculations** lib/
- **state management مرکزی** stores/
- **reusable logic برای components** hooks/

مزایا:

- Testability بالا (logic جدا از UI)
- Reusability (hooks قابل استفاده در چند component)
- Maintainability (تغییر logic بدون دست زدن به UI)

Routing Strategy 🌐

App Router (Next.js 14)

/	Landing page
/product/[id]	Dynamic route محصول
/product/[id]/simple	Nested route (fallback)
/showroom/[slug]	SSG brand pages
/store	Static route
/ar	Static route

تصمیمات Routing

1. Dynamic Routes با [id]

چرا؟ محصولات از JSON config می‌خوانند، نه database

مزایا:

- یک component برای همه محصولات
- URL-based state (share link)
- SEO-friendly

2. Static Generation برای Showrooms

چرا SSG؟

- تعداد محدود برندها (10-20)
- محتوا static است
- Performance بهتر از SSR

3. Client-Side Navigation

از `next/link` و `useRouter` برای SPA experience

چرا؟ بدون page reload، smooth transitions

State Management: Zustand vs Context

تصمیم: کی از کدام استفاده کنیم؟

Zustand - Application State

استفاده:

- رنگ محصول
- لایه فعال (frame/cover/soft)
- وضعیت explode
- paint config

چرا Zustand؟

- State بین چندین component share می شود
- نیاز به update از event handlers (button click)
- debugging برای DevTools
- سبک تر از Redux (1KB)

مثال استفاده:

کاربر در `PresentationSheet` رنگ انتخاب می کند
`ProductCover` component باید re-render شود
بدون prop drilling

Context API - Infrastructure State

استفاده:

- Quality tier (desktop/tablet/phone)
- Canvas settings
- WebGL capabilities

چرا Context؟

- کل app نیاز به این settings دارد
- یک بار set می‌شود، کمتر update
- Provider در root layout

useState - Local State ❌

فقط در همان component نیاز است، مثل hover state یک button

جدول تصمیم‌گیری

دلیل	انتخاب	Scenario
Shared state	Zustand	رنگ محصول بین 5 component
Infrastructure	Context	Quality settings در root
Local	useState	Hover state یک button
High-frequency (60fps)	useRef	Animation progress

Custom Hooks - چرا و کجا؟ 📌

Pattern: استخراج Logic از Components

1. useClipWipe - Clipping Plane Animation

مشکل: Component باید هر frame clipping plane update کند

راه حل: Hook با `useFrame` از R3F

چرا Hook؟

- Logic پیچیده (matrix math)
- قابل استفاده در چند layer
- Test کردن راحت تر

2. useZonePaint - Color Interpolation

مشکل: تغییر رنگ فوری ugly است

راه حل: Lerp با damping

چرا Hook؟

- هر zone (wood/cover/cushion) نیاز دارد
- `useRef` برای 60fps بدون re-render
- Reusable logic

3. useAssetProbe - Asset Pre-check

مشکل: 404 در white screen = GLTF

راه حل: Pre-flight check قبل از Canvas

چرا Hook؟

- Async logic (fetch)
- Loading/error states
- Component stays clean

مقایسه: با Hook vs بدون Hook

با Hook: ✓

5 خط، clean و testable
Logic قابل استفاده مجدد

بدون Hook: ✗

150 خط logic پیچیده در component
غیرقابل test و reuse

React Patterns استفاده شده

1. Server Components vs Client Components

Server Components (پیش فرض)

مزایا:

- Bundle size کوچکتر
- Data fetching روی server
- SEO بهتر

Client Components (با 'use client')

چه موقع Client Component؟

نیاز به browser APIs (window , canvas)
Event handlers (onClick, onHover)
React hooks (useState, useEffect)
Three.js (حتماً client)

2. SSR Disable با Dynamic Imports

مشکل: Three.js در SSR crash می کند

مزایا:

- Bundle splitting
- فقط client load می شود
- Initial page load سریعتر

3. Error Boundaries

منطق: یک layer خراب = کل scene خراب نشود

سناریو:

- `cover.glb` خراب است → error نشان بده
- `frame.glb` و `soft.glb` همچنان render شوند

4. Memo برای Performance

چرا؟

- Parent component (Canvas) هر frame re-render می شود
- Room setup static است
- بدون memo = هر frame rebuild scene

5. useRef برای High-Frequency State

مشکل: `useState` → 60 re-render/sec → re-render

چرا useRef؟

- Animation smooth (هر frame update)
- بدون re-render overhead
- Performance بهتر

Next.js Features استفاده شده 🚀

1. App Router (نه Pages Router)

چرا App Router؟

- Server Components
- Layout nesting
- Route handlers
- Streaming با Suspense

2. Metadata Generation

مزیت: dynamic SEO بدون third-party library

3. Static Asset Optimization

منطق:

- GLB files immutable هستند
- Browser cache برای 1 سال
- Re-deploy = نام فایل تغییر می‌کند

4. TypeScript Integration

همه جا **type-safe**: Props, Config files, Store state, API responses

تصمیمات کلیدی معماری 💡

1. چرا Layered Assets؟

قبل:

یک `product.glb` = 50MB

بعد:

`soft.glb` + `frame.glb` + `cover-red.glb` = 15MB

مزایا:

- تغییر `cover` بدون `reload frame`
- Memory usage پایین
- Parallel loading

2. چرا Demand Rendering؟

قبل:

60fps همیشه = battery drain

بعد:

animation فقط حین idle, 60fps

کی invalidate می‌کنیم؟

- User interaction (drag, click)
- Color animation در حال اجرا
- Layer transition

3. چرا Quality Tiers؟

Effects	Device
N8AO + Bloom + MSAA	Desktop
SSAO + SMAA	Tablet
فقط SMAA	Phone

منطق:

- iPhone 15 $\neq$ iPhone 8
- Automatic downgrade بعد از crash
- User experience > visual fidelity

4. چرا Pre-built AR Assets؟

بعد:

Pre-built GLB/USDZ $\rightarrow$ stable

قبل:

Three.js $\rightarrow$ Runtime export از crash

دلیل:

- Mobile memory محدود
- Serialization expensive
- Production stability

Performance Optimizations 🇮🇷

Code Splitting .1

Three.js برای Dynamic import
Route-based splitting automatic
Lazy load postprocessing effects

Asset Optimization .2

(50-80% کاهش) Draco compression
Progressive loading (frame → cover → soft)
Texture compression

Rendering Optimization .3

Frustum culling (Three.js)
Demand rendering
Adaptive DPR
Memo برای static subtrees

State Optimization .4

animations برای useRef
selective subscriptions با Zustand
expensive operations برای Debounce

چگونه ارائه کنیم؟ 🎤

1. شروع با Problem Statement (2 دقیقه)

"فروشگاه‌های مبلمان مشکل دارند: مشتری باید حضوری بیاید. ما یک showroom مجازی ساختم با 3D و AR."

2. Tech Stack Justification (3 دقیقه)

Next.js App Router - چرا؟ SSG + Server Components

Redux نه Zustand - چرا؟ سبک‌تر، همین کافی بود

Three.js - چرا؟ وب‌سایت، نه native app

TypeScript - چرا؟ Production code needs safety

3. Architecture Deep Dive (5 دقیقه)

Folder structure چرا این طوری؟

State management strategy (Zustand vs Context vs useState)

Routing strategy (SSG vs dynamic)

Component patterns (Server vs Client)

4. یک Challenge + Solution (3 دقیقه)

Option A: Mobile AR Crashes

- مشکل: Runtime export → OOM
- راه‌حل: Pre-built assets
- نتیجه: Zero crashes

Option B: Battery Drain

- مشکل: 60fps always
- راه‌حل: Demand rendering
- نتیجه: 60x reduction

Option C: Visual Jank

- مشکل: Color jumps
- راه حل: Lerp با useRef
- نتیجه: Smooth 60fps

5. Demo (2 دقیقه)

نشان بده live product viewer

تغییر رنگ

Layer transition

AR preview (اگر موبایل داری)

6. Q&A

آماده باش برای:

- "چرا Zustand نه Context API؟"
- "SSR یا CSR؟"
- "Performance در mobile؟"
- "Scalability؟"

سوالات متداول + پاسخ‌های آماده

Q: چرا Zustand به جای Redux؟

A: Redux برای complex async flows عالی است. این پروژه state ساده دارد (رنگ، explode، layer). Zustand 1KB است، Redux 8KB. کمتر boilerplate، همان قابلیت DevTools.

Q: SSG یا SSR یا CSR؟

A:

- **SSG** برای showroom pages (static content)
- **CSR** برای 3D Canvas (نیاز به browser APIs)
- **Hybrid** در `/product/[id]` (CSR برای metadata، Canvas برای SSR)

Q: Performance در mobile چگونه است؟

A: Quality tier system. iPhone 8 با settings پایین render می‌کند، iPhone 15 با full quality. Context loss با detection برای demand rendering. crash recovery برای battery.

Q: چگونه scale می‌کند؟

A:

- محصولات از JSON config (add کردن راحت)
- Components reusable (car configurator همان pattern)
- Folder structure modular (add feature = add folder)

?Q: Security considerations

asset برای XSS، CORS برای resolvers، CSP headers در TypeScript، Input validation با **A:** Type safety loading

Key Takeaways برای مصاحبه‌گر 🎯

Modern React/Next.js Mastery ✅

App Router, Server Components, Dynamic Imports

Thoughtful Architecture ✅

Separation of concerns, Config-driven design, واضح, State management strategy

Performance-First Mindset ✅

Demand rendering, Code splitting, Quality tiers

Production Ready ✅

Error handling, Type safety, Mobile optimization

Problem Solving ✅

measurable results و Practical solutions با Real-world challenges (memory, battery, crashes)

موفق باشی! این پروژه رو با اعتماد به نفس ارائه بده 🚀